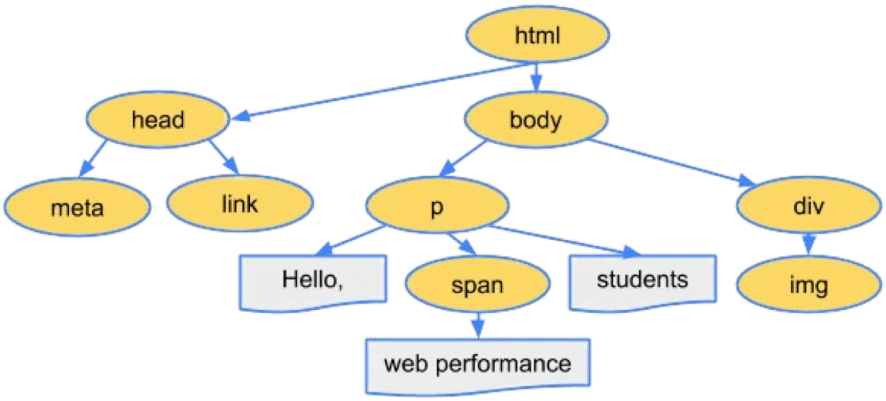
Optimalizace webu - cache, CDN, načítání webu

Načítání webu

- **Prohlížeč při načítání stránky provádí řadu kroků:**

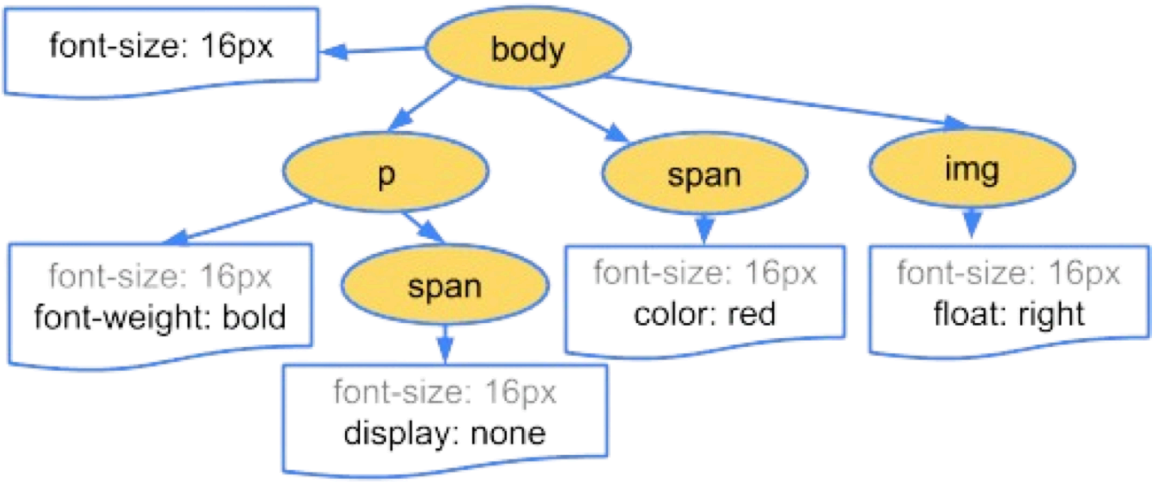
1. **Zadání URL a odeslání HTTP požadavku:**
   - DNS lookup (překlad doménového jména)
     - Proces trvá několik milisekund, proto se ukládá do cache
   - Prohlížeč odešle HTTP požadavek
2. **Přijetí HTTP odpovědi s HTML dokumentem:**
   - Server odpoví HTTP odpovědí, která obsahuje stavový kód
3. **Parsování HTML a vytvoření DOM stromu:**
   - Prohlížeč obdrží HTML dokument jako tok bajtů
   - <u>Parsování probíhá v několika fázích:</u>
     1. **Tokenizace**:
        - Rozdělení textu na tokeny (značky, textové uzly, komentáře)
     2. **Syntaktická analýza:**
        - Vytváření stromové struktury z tokenů
     3. **Vytvoření DOM stromu:**
        - Objektová reprezentace dokumentu v paměti
   - Parsování může být přerušeno skripty



Příklad DOM stromu

4. **Parsování CSS a vytvoření CSSOM stromu:**

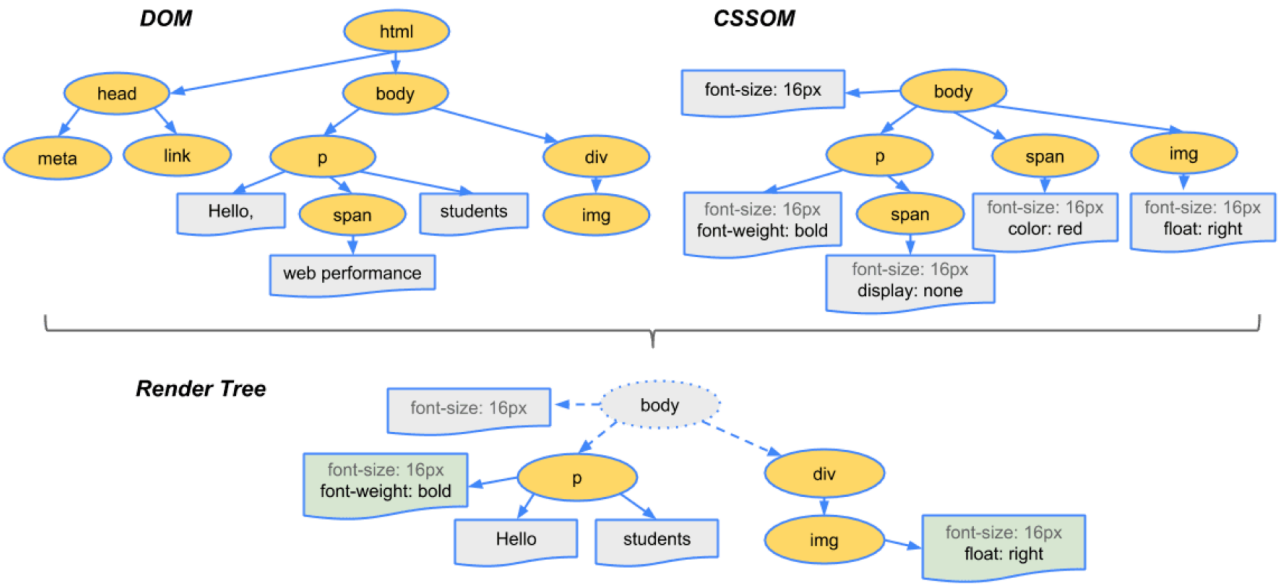
- Prohlížeč načítá CSS soubory definované v HTML:
  - **<link rel=„stylesheet“ href=„style.css“>**
  - **<style>** v dokumentu
- CSS parsování zahrnuje:
  1. **Tokenizace CSS kódu:**
     - Rozdělení na selektory, deklarace, hodnoty
  2. **Syntaktickou analýzu:**
     - Vytvoření stromu pravidel (**CSSOM** - CSS Object Model)
- **CSSOM** představuje strukturu všech **CSS** pravidel aplikovaných na dokument
- Prohlížeč také řeší dědičnost stylů



Příklad CSSOM stromu

5. Konstrukce renderovacího stromu (Render Tree)

- Renderovací strom je vytvořen kombinací DOM a CSSOM stromů
- Obsahuje pouze viditelné elementy
  - Skryté elementy (např. **display: none**) nejsou zahrnuty
- <u>Každý uzel v renderovacím stromu obsahuje:</u>
  - Vizuální informace o uzlu (styly, geometrie)
  - Odkaz na obsah (text, obrázky)
- <u>Proces konstrukce zahrnuje:</u>
  1. Procházení **DOM** stromu a určení viditelnosti uzlů
  2. Přiřazení stylů z **CSSOM** ke každému uzlu
  3. Vytvoření renderovacích objektů (**Render Objects**)



Příklad renderovacího stromu

6. **Layout (reflow):**

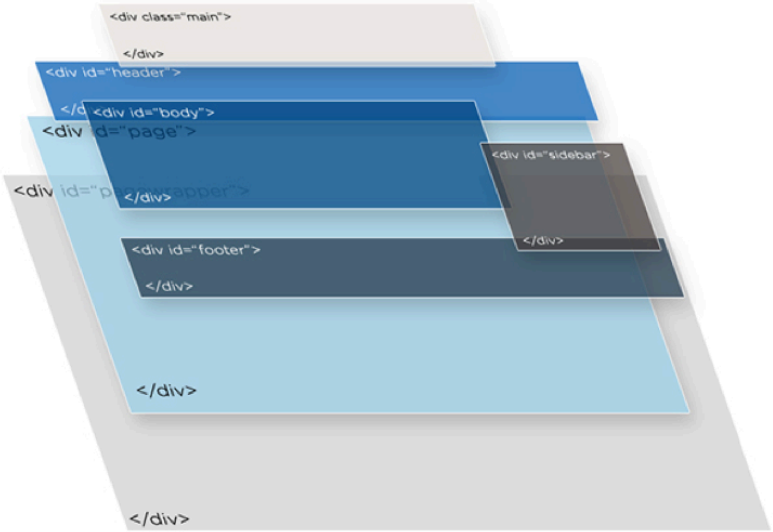
- Proces výpočtu geometrických vlastností každého uzlu v renderovacím stromu
- Zahrnuje určení:
  - Pozice (x, y) vůči nadřazenému elementu
  - Velikosti (šířka, výška) na základě obsahu a stylů
- Layout začíná od kořenového uzlu a postupuje rekurzivně
- Faktory ovlivňující layout:
  - **Box model** (content, padding, border, margin)
  - **Flow model** (block, inline, float, flex, grid)
  - Media queries a responsivní design
- Layout je výpočetně náročný a jeho opakování (reflow) může ovlivnit výkon

7. **Vykreslování (painting):**

- Proces převodu renderovacího stromu na vizuální reprezentaci (pixely)
- Každý uzel je vykreslen podle svých stylů
  - Barvy, pozadí, rámečky, stíny, atd.
- Vykreslování může být rozděleno do několika vrstev (layers)
- Prohlížeč může optimalizovat vykreslování pomocí malování jen změněných oblastí
- Vykreslování je ovlivněno:
  - Komplexností stylů (např. náročné CSS efekty)
  - Velikostí a počtem obrázků
  - Použitím canvas nebo svg grafiky

8. **Kompozice (composition):**

- Skládání jednotlivých vrstev (layers) do finálního obrazu na obrazovce
- Využívá GPU akceleraci pro lepší výkon
- <u>Kompozitní vrstvy mohou obsahovat:</u>
  - Elementy s transformacemi (transform)
  - Elementy s průhledností (opacity)
  - Animované elementy (CSS animace, přechody)
- <u>Proces kompozice:</u>
  1. Každá vrstva je vykreslena do textury v paměti GPU
  2. Textury jsou kombinovány podle pořadí a efektů
  3. Výsledný obraz je zobrazen na obrazovce
- Optimalizace kompozice může výrazně zlepšit výkon animací



Příklad vrstev v CSS

Cache (mezipaměť)

- Ukládá kopie dříve stažených zdrojů (HTML, CSS, JS, obrázky), aby je při opakovaném přístupu nebylo nutné znovu stahovat ze serveru
- Výrazně to snižuje dobu načítání a zatížení serveru
- **Browser cache (mezipaměť prohlížeče):**
  - Prohlížeč si ukládá stažené soubory na disk
  - Při dalším požadavku na stejný zdroj se nejprve podívá do své cache
  - Pokud tam zdroj je, použije ho a přímo bez kontaktování serveru
- **HTTP cache hlavičky:**
  - **Server řídí chování cache pomocí HTTP hlaviček v odpovědi:**
    - Cache-Control:
      - určuje, jak a jak dlouho se má zdroj cachovat
      - **max-age=3600** -> cache je platná 3600 sekund
      - **no-cache** -> zdroj se muže cachovat, ale před každým použitím se musí ověřit u serveru
      - **no-store** -> zdroj se nesmí cachovat
      - **public** -> může cachovat kdokoli
      - **private** -> cachovat smí jen prohlížeč uživatele, ne sdílená cache

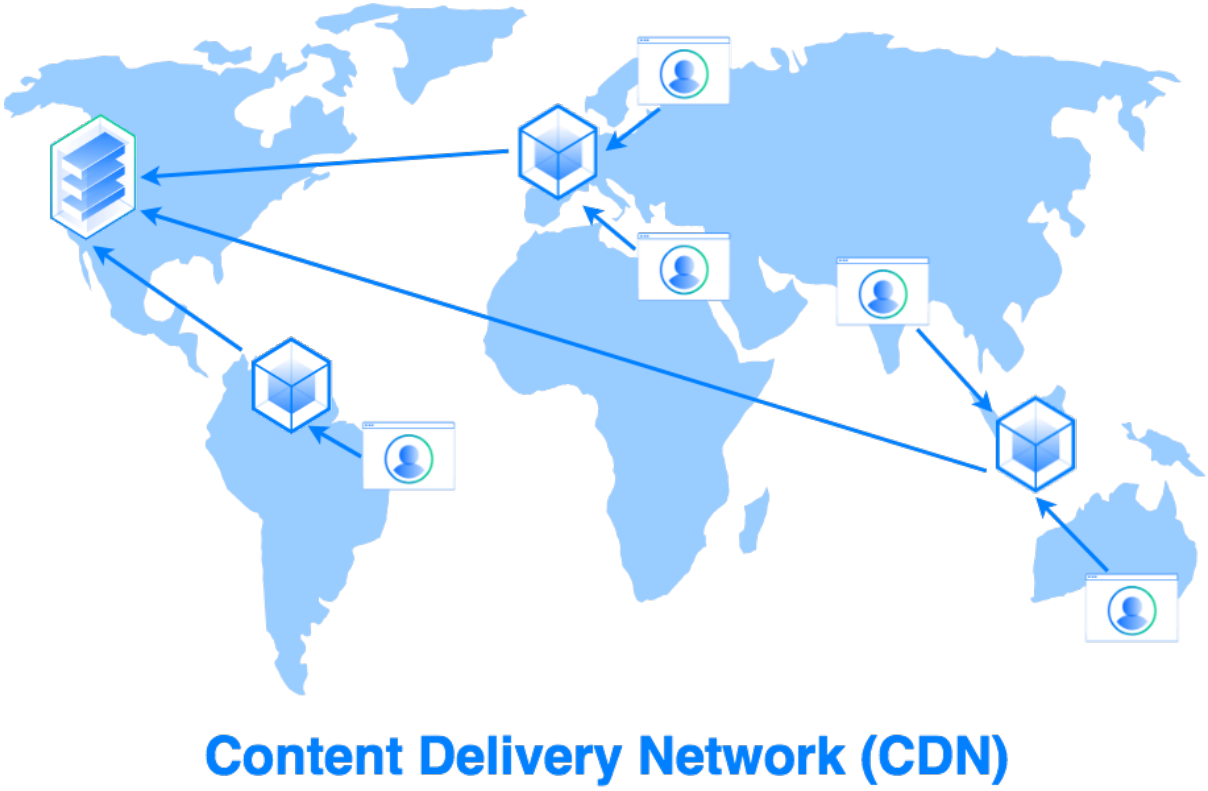
```
HTTP/1.1 200 OK
Content-Type: text/css
Cache-Control: public, max-age=31536000
```

- **CDN cache:**
  - Když uživatel požádá o zdroj, tak edge server se nejprve podívá do své cache
  - Ušetří čas přenosu
- **Service Worker cache:**
  - Je JavaScriptový skript běžící na pozadí v prohlížeči nezávisle na stránce
  - Umožňuje uživateli plně kontrolovat cachování
  - Lze definovat, které zdroje se mají cachovat a jak se mají obsluhovat požadavky
  - **Strategie:**
    - cache-first
    - network-first
    - state-while-revalidate
- **Cache busting:**
  - Problém s cache je, že prohlížeč může zobrazovat zastaralou verzi souboru
  - Řešení je cache busting - přidání verze nebo hasne do názvu souboru, např. style.abc123.css
  - Když se soubor změní, změní se i jeho hash, prohlížeč ho považuje za nový zdroj a stáhne aktuální verzi

CDN (Content Delivery Network)

- Je síť geograficky distribuovaných serverů (tzv. edge routerů)
- Jsou vzájemně propojeny skrze internet
- Zvyšuje dostupnost dat ukládáním obsahu do mezipaměti v blízkosti koncových uživatelů
- **Tuto síť tvoří:**
  - Zdrojový server, který poskytuje obsah pro **CDN**
  - Prvky rozmístěné (**edge routery**) v různých částech internetové infrastruktury, kde je obsah replikován a odkud se doručuje uživatelům
  - Směrovací systém, který zajišťuje, aby byl obsah uživateli doručen z geograficky nejbližšího uzlu
- **CO řeší?:**
  - Když uživatel v Praze pošle požadavek na web v NewYorku, tak bez CDN je to obrovská latence vzhledem k vzdálenosti. (100 - 200 ms na jeden požadavek)
  - CDN to řeší tak, že kopie obsahu umístí na servery po celém světě = uživatel v Praze pak dostane odpověď ze serveru v Frankfurtu nebo přímo v Praze, což sníží latenci na jednotky milisekund
- **Jak funguje:**
  - Provozovatel si nastaví CDN poskytovatele (např. **Cloudflare**). Doména webu se přesměruje tak, aby DNS požadavky procházely přes CDN
  - Uživatel zadá URL do prohlížeči. DNS systém CDN poskytovatele vyhodnotí, odkud uživatel přistupuje a přesměruje požadavek na nejbližší edge server
- **Co doručuje?:**
  - Statické soubory (CSS, JavaScript, obrázky, fonty, videa, PDF)
- **Výhody:**
  - Nižší latence
  - Snížení zátěže origin serveru
  - Lepší dostupnost
  - Ochrana proti DDoS útokům
  - Automatická optimalizace

- **Příklady CDN poskytovatelů:**
  - Cloudflare
  - AWS CloudFront
  - Akamai
  - Fastly
  - Google Cloud CDN



Další optimalizační techniky

- **Lazy loading**: obrázky a jiné zdroje se načítají až ve chvíli, kdy se uživatel k nim přiblíží scrollováním
- **Minifikace**: odstranění mezer, komentářů a zkrácení názvů proměnných v CSS a JS souborech. Zmenší velikost souborů
- **Komprese**: server komprimuje odpověď pomocí gzip. Prohlížeč ji dekomprimuje
- **Preloading a prefetching**:
  - **<link rel=„preload“>**
    - Říká prohlížeči, aby začal stahovat důležitý zdroj co nejdříve
  - **<link rel=„prefetch“>**
    - Představuje zdroje, které budou pravděpodobně potřeba na další stránce
- **Optimalizace obrázků**: použití moderních formátů, správná velikost obrázků pro dané rozlišení a komprese